

DashVoosBrasil

Guia de Apresentação do Projeto

Projeto Final — Banco de Dados Avançado. Este guia explica o projeto de forma didática, sem jargão: o que ele é, como foi montado (a arquitetura), o raciocínio de programação por trás de cada parte e **em qual arquivo cada funcionalidade está** — pensado para uma apresentação a uma banca e para ser entendido por qualquer pessoa.

1. O que é o projeto, em uma frase

É um sistema que pega os dados públicos de **todos os voos comerciais do Brasil** (divulgados pela ANAC), faz uma faxina e uma organização nesses dados, e os apresenta em **dois painéis interativos** no navegador, onde qualquer pessoa pode explorar, filtrar e descobrir padrões da aviação brasileira — são cerca de **3 milhões de voos**, de 2022 ao início de 2025.

O ponto importante para a banca: o projeto não é apenas um gráfico bonito. Ele é um **caminho completo do dado**, da coleta bruta até a informação pronta para decisão.

2. A grande ideia: pense em um restaurante

A melhor forma de entender a arquitetura é imaginar um **restaurante**. O dado cru é o ingrediente que chega do fornecedor; o painel na tela é o prato servido. Entre um e outro existe uma cozinha organizada, com etapas bem separadas. Cada arquivo do projeto é uma estação dessa cozinha:

Estação da cozinha	Arquivo	O que faz
O entregador	coleta_dados.py	Vai até a ANAC e baixa os dados
A faxina e o preparo	prepara_dados.py	Limpa e organiza os ingredientes
A despensa inteligente	lib_dados.py	Enriquece e guarda tudo pronto para uso rápido
O prato executivo	dashboard_visao_geral.py	Painel resumido, visão de diretoria
O prato de degustação	dashboard_exploratorio.py	Painel detalhado, 8 abas para explorar

O raciocínio central por trás disso tem um nome: **separação de responsabilidades**. Cada arquivo faz uma coisa só, e bem feita. Quem baixa não limpa; quem limpa não desenha gráfico. É o que diferencia um projeto amador (tudo num arquivo bagunçado) de um profissional.

3. O caminho do dado, etapa por etapa

Etapa 1 — A coleta (arquivo coleta_dados.py)

A ANAC publica um arquivo para cada mês. Baixar isso na mão seria trabalhoso e sujeito a erro. Então esse arquivo é um **robô coletor** (um crawler): acessa o site automaticamente e baixa cada arquivo mensal. Ele é esperto em três sentidos: se a internet falhar, **tenta de novo** sozinho; se o arquivo **já foi baixado antes**, não baixa de novo; e trata os erros sem quebrar.

Para a banca: essa automação da coleta costuma valer **ponto bônus**, porque mostra que os dados não foram pegos na mão — o sistema se vira sozinho.

Etapa 2 — A faxina e a organização (arquivo prepara_dados.py)

Aqui mora a parte que mais interessa a um professor de Banco de Dados: a etapa clássica de **tratamento de dados** (na indústria, chamada de ETL — Extrair, Transformar, Carregar). Dado da vida real vem sujo, e este arquivo resolve isso, nesta ordem:

- **Juntar tudo.** Pega os 30+ arquivos mensais separados e os empilha num só — como juntar páginas avulsas num único caderno.
- **Ler corretamente.** Arquivos do governo costumam vir com acentos quebrados — o nome 'Galeão', por exemplo, chega escrito com símbolos estranhos no lugar dos acentos. O código detecta o formato certo e conserta automaticamente.
- **Padronizar os nomes.** A ANAC chama uma coluna de 'Sigla ICAO Empresa Aérea'; o código renomeia tudo para nomes curtos e consistentes (EMPRESA, ORIGEM, DESTINO).
- **Calcular o que importa.** A partir do horário previsto e do real, cria informações que não existiam: quantos minutos atrasou, se foi atrasado (regra: +15 min), se foi cancelado, qual a rota, e de qual mês/ano/dia/região o voo é.
- **Jogar fora o lixo.** Remove voos duplicados e dados impossíveis (datas inválidas).

No fim, esse arquivo entrega uma planilha limpa e padronizada (dataset_final.csv).

Para a banca: o código tem até um **plano B**: se os dados reais não estiverem disponíveis, ele gera dados simulados realistas para a demonstração continuar funcionando. Isso é programação defensiva ('e se der errado?').

Etapa 3 — A despensa inteligente (arquivo lib_dados.py)

Este é o **coração técnico** do projeto e o ponto mais forte para defender. Ele resolve dois problemas de uma vez: **velocidade** e **inteligência dos dados**.

Velocidade. A planilha limpa tem 3 milhões de linhas e pesa mais de 1 gigabyte. Se cada painel tivesse que abrir esse arquivo gigante toda vez, demoraria mais de meio minuto e consumiria muita memória. A solução é a ideia mais elegante do projeto: **fazer o trabalho pesado uma vez só**. Este arquivo lê o dado gigante uma única vez, processa tudo e salva uma versão compacta e otimizada (formato Parquet). Os painéis passam a ler essa versão pronta. Resultado real:

- abertura caiu de **~33 segundos para ~0,3 segundo** (mais de 100× mais rápido);
- memória usada caiu de **~2,6 GB para ~0,7 GB**.

É como ter uma despensa onde os ingredientes já vêm picados e temperados: na hora de cozinhar, é instantâneo. E o sistema só refaz esse preparo se o dado original mudar.

Inteligência dos dados. O dado bruto fala em código; este arquivo traduz para humano e adiciona camadas de conhecimento que o original não tinha:

- traduz o código da companhia (AZU) para o nome (Azul) e agrupa por bloco econômico;
- a partir do modelo do avião, descobre o fabricante (Airbus, Boeing, Embraer, ATR);
- calcula a **distância de cada rota** usando as coordenadas geográficas;
- calcula se o voo, depois de sair atrasado, **recuperou tempo no ar**;
- resolve um detalhe técnico: os voos usam um tipo de código de aeroporto (ICAO, 'SBGR') e as tarifas usam outro (IATA, 'GRU') — o arquivo **faz a tradução entre os dois** para conseguir cruzar as bases.

Ele guarda também o **padrão visual** (cores, fontes) e a **formatação em português** (escrever '1.234.567' e 'R\$ 559' no padrão brasileiro). Por que num só lugar? Porque os dois painéis usam este mesmo arquivo: ficam visualmente idênticos, e mudar uma cor se faz num ponto só. Esse princípio se chama '**não se repita**' (DRY).

Frase de efeito: 'Esse módulo é a fundação compartilhada: transforma dado bruto em dado inteligente e o serve pronto e rápido para os dois painéis.'

Etapa 4 — Os dois painéis

Com o dado limpo, enriquecido e rápido, faltam as telas. São dois painéis porque atendem a **dois tipos de pergunta diferentes**:

Painel 1 — Visão Geral (dashboard_visao_geral.py) é o painel de diretoria: a visão de cima. Em 5 segundos você entende o setor — total de voos, pontualidade, cancelamentos, quem domina o mercado, o mapa das rotas pelo Brasil, quais aviões mais voam. Tem um filtro simples (o ano) e, ao escolher um ano, mostra a **variação em relação ao ano anterior** (a seta verde/vermelha), como num relatório financeiro.

Painel 2 — Exploração (dashboard_exploratorio.py) é o painel do analista: para mergulhar fundo. Tem uma **barra lateral de filtros** e **8 abas temáticas**, cada uma respondendo uma pergunta: Visão (rotas e volume), Pontualidade (quando e por que atrasa), Frota & Capacidade, Malha & Geografia (o mapa e o fluxo entre regiões), Tarifas (preços), Curiosidades & Correlações (os fatos surpreendentes), Comparativo (o usuário monta o próprio gráfico) e Tabela (os dados crus, com botão de **baixar em CSV/Excel**).

4. A lógica que faz os painéis reagirem

Quando você mexe num filtro — por exemplo, desmarca a companhia 'Gol' — **todos os gráficos se atualizam sozinhos** na hora. Como? A lógica se chama **programação reativa** e funciona como uma receita de 'quando isso, faça aquilo':

A regra: 'Quando o usuário mudar qualquer filtro, pegue só os voos que correspondem a essa escolha, recalcule os gráficos e redesenhe a tela.'

No código, isso é uma função chamada *callback*. Existe uma função central — em ambos os painéis — chamada **filtrar**, que é o porteiro: recebe as escolhas do usuário e devolve **apenas o pedaço dos dados** que interessa. Os gráficos são sempre desenhados a partir desse pedaço filtrado. Por isso

tudo se mantém coerente: mudou o filtro, muda o pedaço, mudam os gráficos.

- **Eficiência:** no painel de exploração, cada aba só calcula seus gráficos quando você realmente a abre. Se nunca abrir 'Tarifas', o computador nem perde tempo com ela.
- **Diferencial:** os textos de insights e os cartões de curiosidades **não são fixos** — são recalculados a partir do que está filtrado. Não é texto decorado; é o sistema lendo os próprios dados e contando o que encontrou.

5. Mapa rápido: se a banca perguntar X, está em Y

Se perguntarem...	A resposta está em...
De onde vêm os dados?	coleta_dados.py (robô que baixa da ANAC)
Como limpam os dados sujos?	prepara_dados.py (faxina, padronização, duplicados)
Como ficou rápido com 3 milhões de linhas?	lib_dados.py (o cache Parquet)
Como cruzaram voos com tarifas?	lib_dados.py (tradução ICAO e IATA)
Onde estão os indicadores e o mapa?	dashboard_visao_geral.py
Onde está a análise detalhada e os filtros?	dashboard_exploratorio.py
Os gráficos atualizam sozinhos?	Sim — função filtrar + callbacks em cada painel

6. As sacadas do projeto (o que falar para ganhar pontos)

- **Pipeline completo e organizado** — cobre o ciclo inteiro (coleta, limpeza, enriquecimento, visualização), com cada etapa isolada em seu arquivo.
- **Engenharia de dados de verdade** — o cache que deixou tudo 100× mais rápido e a tradução entre os dois padrões de código de aeroporto mostram preocupação real com performance e integração de bases.
- **Dados que viram informação** — o sistema calcula correlações e curiosidades sozinho (ex.: voo curto custa cerca de 18× mais por km que voo longo; a partir de 2024 os voos do Santos Dumont migraram para o Galeão).
- **Robustez** — trata acentos quebrados, dados ausentes, tem plano B com dados simulados e mostra mensagem amigável quando um filtro não retorna nada.

7. Roteiro sugerido de apresentação (8 a 10 minutos)

- **Abertura (30s):** 'Analisamos 3 milhões de voos do Brasil e transformamos isso em dois painéis interativos.' Mostre o Painel 1 já aberto.
- **O caminho do dado (2 min):** conte a história do restaurante — coleta, faxina, despensa rápida, painéis.
- **A sacada técnica (1,5 min):** explique o cache (33s para 0,3s) e a tradução ICAO/IATA. É aqui que você ganha a banca.

- **Demonstração ao vivo (3 min):** mexa num filtro e mostre tudo reagindo; abra a aba Curiosidades e leia 2 ou 3 fatos; mostre o mapa.
- **Fechamento (1 min):** 'Não entregamos um gráfico; entregamos um caminho do dado bruto à decisão.'

Para rodar: `python dashboard_visao_geral.py` (porta 8050) e `python dashboard_exploratorio.py` (porta 8051). A primeira execução monta o cache (~30s); depois cada painel sobe em ~1s.